Functions

24 August 2026 18:18

Quite often we need to perform a similar action in many places of the script. For example, we need to show a nice - looking message when a visitor logs in, logs out and maybe somewhere else.

Functions are the main **building blocks** of the program. They allow the code to be called many times without repetition.

We have already seen example of build - in function like console.log(), prompt(), and confirm(question). But we can create function of our own as well.

Function Declaration
To create a function, we can use a function declaration.
It look like this:

```
function showMessage(){
   console.log(`Hello everyone!`);
}
```

The function keyword goes first, then goes the name of the function, then a list of parameters between the parentheses (comma-separated, empty in the example above)

Syntax:

```
function functionName(parameter1, parameter2, …parameterN){
  //body
 }
```

Our new function can be called by its name, showMessage().

For instance:

```
function showMessage(){
    console.log(`Hello everyone!`)
}
showMessage();
showMessage();
```

Output:

```
Hello everyone!
Hello everyone!
```

The call **showMessage()** executes the code of the function. Here we will see the message two times. This example clearly demonstrate one of the main purpose of function to avoid code duplication. If we ever need to change the message or the way it is shown, it's enough to modify the code in one place, the function which outputs it.

Local variables: A variable declared inside a function is only visible inside that function. For example:

```
function showMessage(){
    let msg = `Hello, I'm JavaScript!`
    console.log(msg)
}
showMessage();
showMessage();
console.log(msg); //<--Error! The variable is local to the function
```

```
Output:
console.log(msg);
            ^

ReferenceError: msg is not defined
    at Object.<anonymous>
```

Outer variable

A function can access an outer variable as well, for example:

```
let userName = `John`;
function showMessage(){
    let msg = `Hello, I'm JavaScript! ` + userName;
    console.log(msg)
}
showMessage();
showMessage();
```

```
Output:
   Hello, I'm JavaScript! John
   Hello, I'm JavaScript! John
```

The function has full access to the outer variable. It can modify it as well.

For instance:

```
let userName = `John`;
function showMessage(){
    userName = `Bob`;
    let msg = `Hello, I'm JavaScript! ` + userName;
    console.log(msg)
}
showMessage();
showMessage();
console.log(userName)
```

Output:

```
Hello, I'm JavaScript! Bob
Hello, I'm JavaScript! Bob
Bob
```

The outer variable is only used if there is no local one.
If the same-named variable is declared inside the function, then it shadows the outer one. For instance, in the code below, the function uses the local username. The outer one is ignored.

```
let userName = `John`;
function showMessage(){
    let userName = `Bob`;
    let msg = `Hello, I'm JavaScript! ` + userName;
    console.log(msg)
}
showMessage();
showMessage();
console.log(userName)
```

Output:

```
Hello, I'm JavaScript! Bob
Hello, I'm JavaScript! Bob
John
```

Global variables: Variables declared outside of any function, such as the outer userName in the code above, are called global.
Global variables are visible from any function (unless shadowed by locals)
It's a good practice to minimize the use of global variable. Modern code has few or no global. Most variables reside in their functions. Sometimes though, they can be useful to store project level data.

Parameters
If we can pass arbitrary data to function using parameters.
In the example below, the function has two parameters: from and text.

```
function showMessage(from, text){
    console.log(`${from} : ${text}`)
}
showMessage('Deeptanarayan', 'Hello!');
showMessage(`Shiv`, 'Good morning!');
```

Output:
   Deeptanarayan : Hello!
   Shiv : Good morning!

Here's one more example, we have a variable **from** and pass it to the function. Please note: The function changes from, but the change is not seen outside because the function always gets a copy of the value:

```
function showMessage(from, text){
    from = `*` + from + `*`;
    console.log(`${from} : ${text}`)
}
let from = `Deeptanarayan`;
showMessage(from, 'Hello!');
// The value of 'from' is the same. The function modified a local copy.
console.log(from)
```

Output:
   Deeptanarayan : Hello!
   Deeptanarayan

When a value is passed as a function parameter, it is also called an argument.
In other words, to put these terms, straight:

- A parameter is the variable listed inside the parenthesis in the function declaration (It's a declaration time term).
- An argument is the value that is passed to the function when it is called (It's a call time term)

We declare function listing their parameters, then call them passing arguments.
In the example above, one might say: The function show message is declared with two parameters, then called with two arguments **from** and **'hello'**.

Default values

If a function is called but an argument is not provided, then the corresponding value become undefined. For instance, the aforementioned function **showMessage(from, text)** can be called with a single argument.

```
showMessage(`Shiv`);
```

That's not an error. Such a call would output *Shiv* : undefined. As the value for text isn't passed, it becomes undefined.

```
function showMessage(from, text){
    from = `*` + from + `*`;
    console.log(`${from} : ${text}`)
}
let from = `Deeptanarayan`;
showMessage(from);
// The value of 'from' is the same. The function modified a local copy.
console.log(from)
```

Output:

```
*Deeptanarayan* : undefined
Deeptanarayan
```

We can specify the so called default (to use. If omitted) value for a parameter in a function declaration using = :

```
function showMessage(from, text=`no text given`){
    from = `*` + from + `*`;
    console.log(`${from} : ${text}`)
}
let from = `Deeptanarayan`;
showMessage(from);
showMessage('Harry', 'Good Afternoon')
```

Output:

```
*Deeptanarayan* : no text given
*Harry* : Good Afternoon
```

Now if text parameter is not passed it will get the value *"no text given"*.

The default value also jumps in if the parameter exist, but strictly equals undefined like this.
Here *"no text given"* is a string. But it can be more complex expression which is only evaluated and assigned if the parameter is missing. So this is also possible:

```
function showMessage(from, text = anotherFunction()){
   //anotherFunction() only executed if no text given
   // its result becomes the value of the text
}
```

Evaluation of default parameters

In Javascript, a default parameter is evaluated every time the function is called without the respective parameter.
In the example above, anotherFunction() isn't called at all if the text parameter is provided.
On the other hand, its independently called every time when text is missing.

Returning a value

A function can return a value back into the calling code as the result.
The simplest example would be a function that sums two values:

```
function sum(a, b){
    return a + b;
}
let result = sum(3, 4)
console.log(result)
```

Output:

```
7
```

The directive return can be in any place of the function. When the function reaches it, the function stops and the value is returned to the calling code. (Assigned to result above).

There may be many occurrences of return in a single function, for instance:

```
function checkAge(age){
    if(age >= 18){
        return true;
    }
    else{
        return false;
    }
}
const prompt = require('prompt-sync')();
let age = prompt('How old are you?', 18);
if(checkAge(age)){
```

```
        console.log('Access granted');
    }
    else{
        console.log(`Access denied`);
    }
```

Output:

How old are you?16
Access denied

It is possible to use *return* without a value. That causes the function to exit immediately.

```
function checkAge(age){
    if(age >= 18){
        return true;
    }
    else{
        return false;
    }
}
function showMovie(age){
    if(!checkAge(age)){
        return;
    }
    console.log(`Showing you the movie`);
}
const prompt = require('prompt-sync')();
let age = prompt('How old are you?', 18);
showMovie(age);
```

Output:

1. How old are you?18
   Showing you the movie

2. How old are you?16

In the code above, if checkAge(age) returns false, then show movie won't proceed to the console.log()

Naming a function

Functions are actions, so their name is usually a verb. It should be brief, as accurate as possible, and describe what the function does so that someone reading the code gets an indication of what the function does.

It is a widespread practice to start a function with a verbal prefix which vaguely describes the action. There must be an agreement within the team on the meaning of the prefixes.

For instance, function that starts with '*show*' usually show something.

Note: *Function should be sort and do exactly one thing. If that thing is big, maybe. It worth it to split the function into a smaller functions. Sometimes following this rule may not be that easy, but its definitely a good thing. A separate function is not only easier to test and debug - it's very existence is a great comment.*

Homework:

JavaScript Practice: Function Basics

Seven practice problems covering function declarations, local vs. outer variables, parameters, default values, return values, naming conventions, and single-responsibility functions.

Question 1 — Local Scope and Shadowing

Write a function `trackTemperature()` that declares a local variable `reading` set to `20`, then logs a message like `"Current reading: 20"`.

- Before calling the function, declare a global variable `reading` set to `100` and log it.
- Call `trackTemperature()` and log the message it produces.
- After the call, log the global `reading` again. Explain in a comment why its value did or did not change.
- Now modify `trackTemperature()` so it does **not** declare a local `reading`, but instead reassigns the outer one directly (e.g. `reading = reading + 5`). Call it again and log the global `reading` to show the difference in behavior between shadowing a variable and modifying it.

Question 2 — Outer Variables and Side Effects

You have a global variable `let cartTotal = 0;`.

- Write a function `addToCart(price)` that adds `price` to `cartTotal` (no `return` statement — it should work purely as a side effect on the outer variable).
- Call `addToCart(25)`, `addToCart(40)`, and `addToCart(15)`, then log the final value of `cartTotal`.
- Rewrite the whole thing a second way: instead of modifying an outer variable, write `addToCart(currentTotal, price)` that takes the current total as a parameter and **returns** the new total, with no reliance on any outer variable.

In a comment, explain which of the two versions is easier to test and reason about, and why the lesson recommends favoring parameters and return values over modifying outer variables.

Question 3 — Parameters, Arguments, and Copying by Value

Write a function `discountPrice(price, discount)` that takes a price and a discount amount, reassigns `price = price - discount` inside the function body, and logs the adjusted price.

Declare a variable `let ticketPrice = 100;` outside the function.
Call `discountPrice(ticketPrice, 30)` and log the result it prints.
After the call, log `ticketPrice` again from the outer scope.
Explain in a comment why `ticketPrice` is unaffected by the reassignment that happened inside the function, connecting your answer to how arguments are copied into parameters.
Bonus: call the function without a second argument, e.g. `discountPrice(ticketPrice)`. What gets logged, and why?

Question 4 — Default Parameters and When They're Evaluated

Write a function `randomId()` that returns a random number between 1 and 1000 (you can use `Math.random()`), and a function `createUser(name, id = randomId())` that logs a message combining `name` and `id`.

Call `createUser("Sam")` twice in a row and log both results. Confirm the `id` is different each time.
Now call `createUser("Sam", 42)` and confirm `randomId()` is *not* invoked in this case (add a `console.log` inside `randomId` to prove it never runs).
Rewrite `createUser` so instead of a default parameter, it manually checks whether `id` is `undefined` inside the function body and calls `randomId()` only then.
Finally, rewrite it once more using the nullish coalescing operator (`??`) instead of an `if` check. In a comment, explain a situation where using `||` instead of `??` for this default would cause a bug (think about what happens if someone passes `id = 0`).

Question 5 — Return Values and Early Exit

Write a function `classifyAge(age)` that:

- Returns the string `"child"` if `age` is under 13.
- Returns the string `"teen"` if `age` is 13–19.
- Returns the string `"adult"` otherwise.

Implement it using multiple `return` statements (no `else` needed after a `return`).
Write a second function `canRide(height, minHeight)` that uses an early,

valueless return to exit immediately if height < minHeight, and otherwise logs "Enjoy the ride!" after the early-return check. Call it with a few different values to demonstrate both paths.
Write a function doNothing() with an empty body, and one more, doNothingExplicit(), containing only return;. Use === comparisons against undefined to prove both return the same thing.

In a comment, explain why the following code is broken, and rewrite it correctly:

```
function getFullName(first, last) {  return    first + " " + last;}
```

Question 6 — Naming Conventions and Prefixes

Design (write full working code for) a tiny "inventory" mini-program made of at least five functions, following the prefix conventions from the lesson:

- A get... function that returns a value (e.g. getItemCount(inventory, itemName)).
- A calc... function that calculates something (e.g. calcTotalValue(inventory)).
- A create... function that creates and returns something new (e.g. createInventory() returning an empty array or object).
- A check... function that checks a condition and returns a boolean (e.g. checkInStock(inventory, itemName)).
- A show... function that is the *only* one of the five allowed to actually log/display anything to the console.

Wire them together in a short script that builds an inventory, adds a couple of items, and prints a summary using only show... for output. In a comment, note which function(s) would violate the "one function — one action" rule if you accidentally added a console.log inside them.

Question 7 — Splitting a Function for Readability ("Functions == Comments")

Below is a single, monolithic function that prints all "Armstrong numbers" (numbers where the sum of each digit raised to the power of the digit count equals the number itself, e.g. 153) up to n, all crammed into one block using a labeled loop:

```
function showArmstrong(n) {
  outer: for (let num = 1; num <= n; num++) {
    let digits = String(num).split('');
    let power = digits.length;
    let sum = 0;
    for (let d of digits) {
      sum += Math.pow(Number(d), power);
    }
```

```
    if (sum !== num) continue outer;
    console.log(num);
  }
}
```

Refactor this into two functions: `isArmstrong(num)`, which does only the check and returns `true`/`false`, and `showArmstrong(n)`, which loops from `1` to `n` and logs the numbers for which `isArmstrong` returns `true` — with no labeled loop needed.

Test both versions (original and refactored) with `n = 1000` and confirm they log the same numbers.

In a comment, explain — in your own words — why the second version is a better "comment" for what the code does, referencing the specific line where the improvement is most visible.

Bonus: split `isArmstrong` even further into a helper `digitCount(num)` and a helper `digitPowerSum(num, power)`, so `isArmstrong` becomes a one-line comparison.